

NAME: S.GAYATHERI

REG NO:192521306

COURSE CODE: CSA0619

COURSE NAME : Design

Analysis of Algorithm

CO5 – AT2 --Pseudocode Writing Task (Algorithm Design)

Q12. Hamiltonian Path Problem – Backtracking Algorithm

1. Problem Understanding

A **Hamiltonian Path** is a path in a graph that visits **every vertex exactly once**.

Input

- A graph $G(V, E)$
- Graph represented using an **adjacency matrix**
- $\text{graph}[i][j] = 1$ if an edge exists between vertices i and j
- $\text{graph}[i][j] = 0$ otherwise

Output

- A valid Hamiltonian Path, if one exists
- Otherwise, return "**No Hamiltonian Path Exists**"

Constraint

Each vertex must:

1. Be visited exactly once.
2. Be connected to the previous vertex in the path.

2. Backtracking Idea

The algorithm builds the path one vertex at a time.

For every possible next vertex:

- Check whether it is adjacent to the previous vertex.
- Check whether it has already been visited.
- If both conditions are true, add it to the path.
- If no valid continuation is possible, **backtrack** and try another vertex.

This pruning avoids exploring invalid paths unnecessarily.

3. Pseudocode

ALGORITHM HamiltonianPath(graph, n)

INPUT:

graph[n][n] // Adjacency matrix

n // Number of vertices

OUTPUT:

Hamiltonian Path if it exists

Otherwise "No Hamiltonian Path Exists"

CREATE path[n]

CREATE visited[n]

FOR $i \leftarrow 0$ TO $n - 1$ DO

visited[i] $\leftarrow$ FALSE

END FOR

FOR start $\leftarrow 0$ TO $n - 1$ DO

path[0] $\leftarrow$ start

visited[start] $\leftarrow$ TRUE

IF FindPath(graph, path, visited, 1, n) = TRUE THEN

PRINT "Hamiltonian Path:"

PRINT path

RETURN

END IF

visited[start] $\leftarrow$ FALSE

END FOR

PRINT "No Hamiltonian Path Exists"

END ALGORITHM

Recursive Backtracking Function

```
FUNCTION FindPath(graph, path, visited, position, n)

    // Base Case:

    // All vertices are included in the path

    IF position = n THEN

        RETURN TRUE

    END IF

    previousVertex ← path[position - 1]

    FOR vertex ← 0 TO n - 1 DO

        // Check adjacency and whether vertex is already visited

        IF graph[previousVertex][vertex] = 1

            AND visited[vertex] = FALSE THEN

                // Choose the vertex

                path[position] ← vertex

                visited[vertex] ← TRUE

                // Recursively find the remaining path

                IF FindPath(graph, path, visited,

                    position + 1, n) = TRUE THEN

                    RETURN TRUE

                END IF

                // Backtrack

                visited[vertex] ← FALSE

            END IF

        END FOR

        RETURN FALSE

    END FUNCTION
```

4. Step-by-Step Working

Suppose the graph has vertices:

$$V = \{0, 1, 2, 3\}$$

Step 1: Select a Starting Vertex

Choose vertex 0.

Path = [0]

Visited = {0}

Step 2: Find an Adjacent Unvisited Vertex

Check all vertices connected to 0.

For example, if vertex 1 is connected:

Path = [0, 1]

Visited = {0, 1}

Step 3: Continue Recursively

Find an unvisited vertex adjacent to 1.

Path = [0, 1, 2]

Visited = {0, 1, 2}

Continue until all vertices are included.

Path = [0, 1, 2, 3]

This is a valid Hamiltonian Path.

Step 4: Backtracking

If no valid vertex can be selected:

Path = [0, 1, 2]

and no unvisited vertex is adjacent to 2, the algorithm removes the last vertex and tries another possibility.

Backtrack:

Path = [0, 1]

This process continues until a valid path is found or all possibilities are exhausted.

5. Pruning Conditions

The algorithm eliminates invalid paths early using the following conditions:

Condition 1: Vertex Already Visited

`visited[vertex] = TRUE`

Such a vertex cannot be selected again.

Condition 2: No Edge Exists

`graph[previousVertex][vertex] = 0`

The vertex cannot be added because it is not connected to the previous vertex.

Therefore, only vertices satisfying:

`graph[previousVertex][vertex] = 1`

AND

`visited[vertex] = FALSE`

are explored.

This reduces unnecessary search.

6. Correctness

The algorithm is correct because:

- It considers every possible starting vertex.
- At each step, it selects only an adjacent vertex.
- The visited array ensures that no vertex is repeated.
- A path is accepted only when exactly n vertices have been included.
- Backtracking ensures that alternative paths are explored when a selected path fails.

Therefore, if a Hamiltonian Path exists, the algorithm can find one. If all possible valid paths fail, it correctly reports that no Hamiltonian Path exists.

7. Time and Space Complexity

Time Complexity

In the worst case, the algorithm may explore all possible vertex arrangements:

$$O(n!)$$

where n is the number of vertices.

Space Complexity

The path array, visited array, and recursive call stack require:

$$O(n)$$

8. Result

The backtracking algorithm successfully determines whether a **Hamiltonian Path** exists by constructing the path vertex by vertex. Invalid paths are pruned using adjacency and visited checks, while backtracking explores alternative choices. The final output is either a valid path containing every vertex exactly once or a message indicating that no Hamiltonian Path exists.